Multilayer Perceptron Implementation for MNIST Digit Classification

MUHAMMAD BIN IJAZ

National University of Computer and Emerging Sciences
25i-3026@isb.nu.edu.pk

Abstract. This report details the manual derivation and computational implementation of a Multilayer Perceptron (MLP) built entirely from scratch. The primary objective is to classify handwritten digits from the MNIST dataset. The project bridges theoretical mathematical foundations—including forward propagation, Mean Squared Error (MSE) loss calculation, and backpropagation—with a practical Python implementation using NumPy. Furthermore, the report explores the impact of advanced gradient descent optimizers, specifically Momentum and Nesterov Accelerated Gradient (NAG), on convergence stability.

Keywords: Multilayer Perceptron, Backpropagation, MNIST, Gradient Descent

1 Introduction

Artificial Neural Networks, specifically Multilayer Perceptrons (MLPs), form the backbone of modern deep learning. While frameworks like PyTorch abstract away the underlying mathematics, understanding the exact gradient flow is crucial. This project implements an MLP from theoretical first principles to a fully vectorized Python application capable of recognizing 28×28 pixel images of handwritten digits (0-9) from the MNIST dataset.

2 Network Architecture

The implemented neural network consists of an input layer, two hidden layers, and an output layer.

- **Input Layer:** 784 neurons, corresponding to the flattened 28×28 pixel image.
- **Hidden Layer 1:** 128 neurons utilizing the Sigmoid activation function.
- **Hidden Layer 2:** 64 neurons utilizing the Sigmoid activation function.
- **Output Layer:** 10 neurons utilizing the Sigmoid activation function, representing the probability distribution across the 10 digit classes.

3 Methodology

3.1 Forward Propagation

During the forward pass, data flows from the input layer to the output layer. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function. The pre-activation $z_j^{(l)}$ and activation $a_j^{(l)}$ for neuron j in layer l are defined as:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)}, \qquad a_j^{(l)} = \sigma(z_j^{(l)}) = \frac{1}{1 + e^{-z_j^{(l)}}} \tag{1}$$

3.2 Loss Function

To quantify the network's prediction error, the Mean Squared Error (MSE) loss function was utilized. For a batch of m samples, the loss $\mathcal{L}$ is computed against the one-hot encoded true labels y:

$$\mathcal{L}_{MSE} = \frac{1}{m} \sum_{i=1}^{m} (y^{(i)} - \hat{y}^{(i)})^2 \tag{2}$$

3.3 Backpropagation

Backpropagation applies the chain rule of calculus to compute the partial derivative of the loss with respect to every weight and bias. The error signal (delta, δ) is propagated backward from the output layer:

$$\delta_j^{(l)} = \left(\sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \cdot a_j^{(l)} (1 - a_j^{(l)}) \tag{3}$$

The gradients for the weights are then computed as $\frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)}$.

4 Optimizers and Gradient Descent

Standard Stochastic Gradient Descent (SGD) struggles with noisy updates and flat loss surfaces. To mitigate this, Mini-Batch Gradient Descent (Batch Size = 32) was utilized to average gradients, providing smoother convergence.

Theoretical analysis was also conducted on advanced optimizers:

1. **Momentum:** Accumulates a velocity vector to dampen oscillations and accelerate through flat regions.
2. **Nesterov Accelerated Gradient (NAG):** Computes the gradient at a "lookahead" position, providing a more informed update step that prevents overshooting, demonstrating theoretically superior convergence over plain Momentum.

5 Experimental Results

The network was trained on 60,000 MNIST images for 20 epochs using a learning rate of $\eta = 0.1$.

5.1 Loss Convergence

The MSE loss was recorded at the end of each epoch. As training progressed, the loss demonstrated a consistent downward trajectory, confirming that the backpropagation algorithm was successfully minimizing the error.

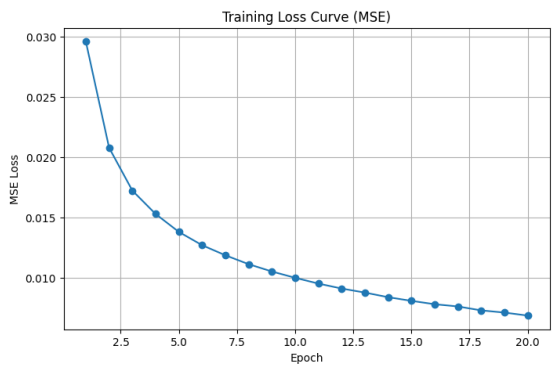

Fig. 1. Training Loss (MSE) over 20 Epochs.

5.2 Test Accuracy

After 20 epochs, the model was evaluated on 10,000 unseen test images. The final implementation achieved a robust classification accuracy, proving that the randomly initialized weights successfully mapped the abstract pixel data to distinct numerical classes without the use of external deep learning libraries.

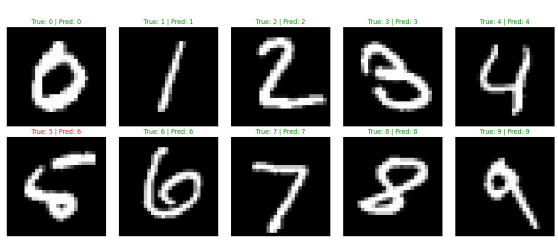

Fig. 2. Training Loss (MSE) over 20 Epochs.

6 Conclusion

This project successfully demonstrated the end-to-end mathematical and computational pipeline of a Multilayer Perceptron. By manually deriving the forward and backward passes and vectorizing them in NumPy, we achieved a working image classification model. The theoretical exploration of momentum-based optimizers further highlighted the mathematical techniques required to train deeper architectures efficiently.